

OBJECT-ORIENTED PROGRAMMING

in Java — Complete Concept Guide & Interview Preparation

Expanded Edition · With Advanced Topics & 45+ Interview Q&A

🔒 Encapsulation

🌀 Abstraction

👉 Inheritance

🔄 Polymorphism

Covers Java 8 – 17 language features · Diagrams, worked examples, and gotchas included

1. What Is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm built around **objects** — bundles of state (data) and behavior (functions) — rather than around a sequence of instructions acting on shared data, as in procedural programming. Java is one of the most widely used OOP languages, and almost everything in it revolves around classes and objects.

OOP makes large programs easier to design, extend, and maintain because it encourages:

- **Modularity** — each class is a self-contained unit that can be built and tested independently.
- **Reusability** — inheritance and composition let you reuse existing code instead of rewriting it.
- **Scalability** — new features can usually be added as new classes without touching old, working code (this is the essence of the *Open/Closed Principle*, covered later).
- **Real-world modeling** — objects map naturally onto real entities (a `Car`, a `BankAccount`, a `Student`), which makes systems easier to reason about.

Interview angle: Java is often called a "partially" object-oriented language because it still has 8 primitive types (`int`, `char`, `boolean`, `byte`, `short`, `long`, `float`, `double`) that are not objects. A "pure" OOP language (like Smalltalk) treats absolutely everything — including numbers — as an object. Java's wrapper classes (`Integer`, `Character`, ...) exist precisely to let primitives participate in the object world when needed (e.g. inside collections).

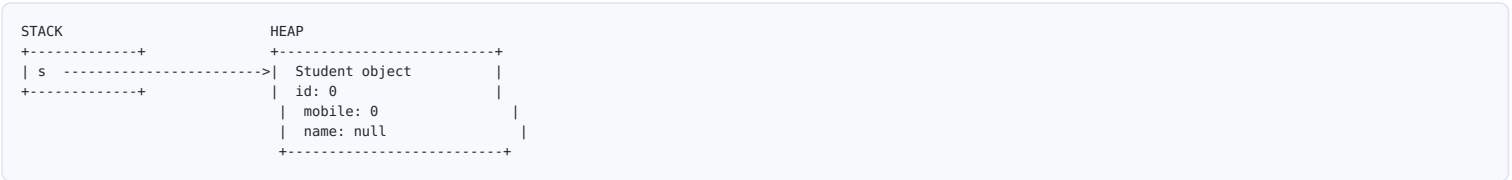
1.1 Class vs Object

A **class** is a user-defined blueprint or template that describes what data (fields) and behavior (methods) its instances will have. It is a purely logical construct — declaring a class does not, by itself, allocate any memory for objects. An **object** is a concrete, run-time instance of a class, created with the `new` keyword, occupying real memory on the heap.

```
class Student {
    public int id;           // instance field (data member)
    public int mobile;
    public String name;

    public int add(int x, int y) { // instance method
        return x + y;
    }
}

Student s = new Student(); // "s" is a reference; the object lives on the heap
```



Note: In Java, objects are *always* created on the heap via `new` — there is no stack-allocated object like in C++. The reference variable (`s`) itself lives on the stack (if it's a local variable) and simply stores the heap address. Java has no `delete`; the JVM's **Garbage Collector (GC)** automatically reclaims memory once an object becomes unreachable. An uninitialized object reference holds the special value `null`.

Advanced: how "unreachable" is decided

The GC doesn't use simple reference counting (which cannot free cyclic structures like two objects pointing at each other). Instead, modern JVM collectors (e.g. G1, ZGC) trace reachability from a set of **GC roots** — local variables on active thread stacks, static fields, and JNI references. Anything not reachable from a GC root, directly or transitively, is eligible for collection, cycles included.

2. Encapsulation

Encapsulation bundles data and the methods that operate on that data into a single unit (a class), and restricts direct outside access to the internal state. Fields are typically declared `private`, and controlled access is exposed through public **getter** and **setter** methods. This is what makes **data hiding** possible — a component can change its internal representation freely as long as its public contract (method signatures) stays the same.

```
class Account {
    private double balance; // hidden from the outside world

    public double getBalance() { // getter
        return balance;
    }

    public void setBalance(double balance) { // setter, with validation
        if (balance >= 0) {
            this.balance = balance;
        } else {
            throw new IllegalArgumentException("Balance cannot be negative");
        }
    }
}
```

Why it matters: Without encapsulation, any part of a large codebase could set `balance = -500` directly, silently breaking invariants. A setter is a single choke-point where you can validate, log, trigger events, or later swap the field for a computed value — all without touching the caller's code.

Encapsulation vs Abstraction — the classic confusion

Encapsulation	Abstraction
Implementation-level: <i>how</i> data is protected (private fields + public methods).	Design-level: <i>what</i> the object exposes conceptually, hiding unnecessary detail.
Achieved with access modifiers.	Achieved with abstract classes and interfaces.
Answers "can outside code touch this field directly?"	Answers "does the caller need to know how this works internally?"

3. Abstraction

Abstraction means modeling a real-world problem by exposing only the essential features and hiding unnecessary implementation details. When you call `list.add(item)` you don't need to know whether the list is backed by an array or a linked structure internally — that complexity is abstracted away. In Java, abstraction is achieved primarily through **abstract classes** and **interfaces** (both covered in depth in Section 8).

Data binding is a related UI concept: it's the mechanism that keeps an application's UI in sync with its underlying business logic (e.g. JavaFX properties, or Android's data-binding library) — a change in the model automatically reflects in the view, and vice versa.

4. Inheritance

Inheritance lets one class (the **subclass** / derived class) acquire the fields and methods of another (the **superclass** / base class) using the `extends` keyword. It models an **"IS-A"** relationship (a `Dog` IS-A `Animal`) and is one of the primary mechanisms for code reuse in Java.

```
class DerivedClass extends BaseClass {
    // inherits all non-private members, can add or override more
}
```

Note: Unlike C++, Java has no "visibility mode" for inheritance (no `private` / `protected` / `public` before the base class name). Every class is inherited in what would be C++'s "public" style; visibility is instead controlled purely by each member's own access modifier.

4.1 Types of Inheritance

Type	Description
Single	One class inherits directly from one parent.
Multilevel	A class inherits from a class that itself inherits from another (A → B → C).
Hierarchical	Multiple classes inherit from the same single base class.
Multiple	A class inherits from two or more classes at once. Not allowed for classes in Java.
Hybrid	A combination of the above patterns.

Why Java forbids multiple class inheritance: the classic **Diamond Problem** — if class `D` extended both `B` and `C` , and both `B` and `C` extended `A` and overrode the same method differently, the compiler would have no unambiguous way to decide which version `D` should inherit. Java sidesteps this entirely by disallowing `extends` with more than one class, and instead achieves multiple-inheritance-like behavior through **interfaces** (a class may `implements` many interfaces). If two interfaces provide conflicting default methods, Java *forces* the implementing class to override the method explicitly and pick a resolution — the ambiguity is caught at compile time, never guessed at runtime.

```
interface Flyer { default void move() { System.out.println("flying"); } }
interface Runner { default void move() { System.out.println("running"); } }

class Superhero implements Flyer, Runner {
    // Conflict! Must override explicitly:
    @Override
    public void move() {
        Flyer.super.move();    // choose explicitly, or write custom logic
    }
}
```

4.2 Composition over Inheritance

Inheritance creates a tight, compile-time coupling: a subclass depends on the internal behavior of its superclass, and changes to the parent can silently break children (the "fragile base class" problem). A commonly cited design guideline — "favor composition over inheritance" — suggests using **aggregation/composition** (a class *holding a reference* to another, a "HAS-A" relationship) instead of inheritance wherever the relationship is not a true, stable IS-A.

```
// HAS-A (composition/aggregation) instead of IS-A
class Engine {
    void start() { System.out.println("Engine starting..."); }
}
class Car {
    private final Engine engine = new Engine(); // Car HAS-A Engine
    void start() { engine.start(); }
}
```

Aspect	Inheritance (IS-A)	Composition (HAS-A)
Coupling	Tight (compile-time)	Loose (can swap implementations at runtime)
Reuse of code	Automatic via extends	Explicit, via delegation
Flexibility	Fixed at compile time (single parent)	Can compose many collaborators
Risk	Fragile base class problem	More boilerplate delegation code

5. Polymorphism

Polymorphism ("many forms") is the ability of a single interface — a method name or reference type — to represent different underlying behaviors depending on context. Java supports two kinds:

- Compile-Time (Static) Polymorphism** — Method Overloading
- Runtime (Dynamic) Polymorphism** — Method Overriding

5.1 Compile-Time Polymorphism: Method Overloading

Overloading means defining multiple methods with the same name in the same class, differentiated by their **parameter list** — either the number of parameters or their types (or both). The compiler resolves which version to call at compile time, based on the static (declared) types of the arguments — this is called **static binding**.

```
class Add {
    public int add(int a, int b)          { return a + b; }
    public int add(int a, int b, int c)   { return a + b + c; }
    public double add(double a, double b) { return a + b; }
}
```

Gotcha (frequently asked): the return type alone is *never* enough to overload a method — the parameter list must differ. `int add(int a){}` and `double add(int a){}` in the same class will not compile: it's a duplicate method signature as far as the compiler is concerned.

Overload resolution order (advanced)

When several overloads could technically match a call, the compiler picks the *most specific* match using three passes: (1) exact match without boxing/unboxing or varargs, (2) match allowing autoboxing/unboxing, (3) match allowing varargs. This is why, given overloads for `int` and `Integer`, a call with a literal `int` binds to the `int` version rather than auto-boxing to `Integer`.

5.2 Runtime Polymorphism: Method Overriding

Overriding happens when a subclass redefines a method that is already defined in its superclass, with the exact same signature. Which version runs is decided at **runtime**, based on the actual object's type — not the type of the reference variable pointing to it. This is called **dynamic method dispatch**.

```
class BaseClass {
    public void show() { System.out.println("Apni Kaksha base"); }
}
class DerivedClass extends BaseClass {
    @Override
    public void show() { System.out.println("Apni Kaksha derived"); }
}
public class Main {
    public static void main(String[] args) {
        BaseClass b;
        DerivedClass d = new DerivedClass();
        b = d;
        b.show(); // reference type is BaseClass, but object type is DerivedClass
    }
}
```

Output: Apni Kaksha derived

Why overriding “just works” in Java: every non-static, non-final, non-private method is “virtual” by default — there is no separate `virtual` keyword as in C++. `@Override` is optional but strongly recommended: it makes the compiler verify you're actually overriding a real superclass method (catching typos in the signature at compile time instead of silently creating an unrelated overload).

How dynamic dispatch actually works under the hood (advanced)

Every class has a hidden per-class structure — informally, a **virtual method table (vtable)** — mapping each virtual method to the code that should run for that class. Each object carries a hidden pointer to its class's method table. When you call `b.show()`, the JVM doesn't look at the compile-time type of `b`; it follows the object's method-table pointer (i.e. looks at the object's *actual* runtime class) and invokes whatever `show()` resolves to there. This is why overriding is resolved at runtime while overloading is resolved at compile time — overloading is decided purely from the static types visible to the compiler; overriding requires knowing which object is actually sitting in memory.

Rules that govern valid overriding

- Method name and parameter list must match *exactly*.
- The return type must be the same, or a **covariant** return type (a subtype of the original return type).
- The access modifier cannot be more restrictive than the overridden method (you can widen `protected → public`, but not narrow `public → private`).
- A method marked `final` cannot be overridden at all.
- A `static` method cannot be overridden — see “Method Hiding” below.
- Constructors are never inherited, so they cannot be overridden.
- The overriding method cannot throw new or broader *checked* exceptions than the overridden method.

```
class Animal {
    Animal reproduce() { return new Animal(); }
}
class Dog extends Animal {
    @Override
    Dog reproduce() { return new Dog(); } // covariant return type: Dog is-a Animal
}
```

Method Overloading vs Method Overriding vs Method Hiding

Aspect	Overloading	Overriding	Hiding (static methods)
Where	Same class (or subclass)	Superclass & subclass	Superclass & subclass
Signature	Must differ (params)	Must be identical	Must be identical
Binding	Compile time (static)	Runtime (dynamic)	Compile time (static)
Resolved by	Argument types at compile time	Object's actual runtime type	Reference's declared type

```
class Parent {
    static void greet() { System.out.println("Parent static"); }
}
class Child extends Parent {
    static void greet() { System.out.println("Child static"); } // hides, doesn't override
}
public class Main {
    public static void main(String[] args) {
        Parent p = new Child();
        p.greet(); // calls based on REFERENCE type -> "Parent static"
    }
}
```

Output: Parent static (compare this with the DerivedClass example above, which prints "derived")

6. Constructors

A constructor is a special block, invoked automatically when an object is created with `new`, typically used to initialize the object's fields. In Java, a constructor has exactly the same name as its class and **no return type** — not even `void` (writing `void` there makes it an ordinary method, not a constructor — a classic interview trick question).

6.1 Types of Constructors

- **Default constructor** — no arguments. If you don't write any constructor, the compiler silently inserts a no-arg default constructor that calls `super()` and leaves fields at their default values (`0`, `false`, `null`).
- **Parameterized constructor** — takes arguments, letting each object be initialized differently.
- **Copy constructor** — Java has no built-in copy constructor like C++. If you need to build an object as a copy of another, you write it by hand (or use `clone()` / a copy factory method).

```
class Go {
    int x;
    Go(int a) { x = a; }           // parameterized
    Go(Go i) { x = i.x; }         // user-defined "copy constructor"
}

public class Main {
    public static void main(String[] args) {
        Go a1 = new Go(20);
        Go a2 = new Go(a1);
        System.out.println(a2.x);
    }
}
```

Output: 20

6.2 Constructor Chaining — `this()` and `super()`

A constructor can call another constructor of the *same* class using `this(...)`, or the immediate parent's constructor using `super(...)`. If used, either call must be the very first statement in the constructor body. If you don't explicitly call `super(...)`, the compiler automatically inserts a call to the parent's no-arg `super()` as the first line.

```
class Vehicle {
    Vehicle() { System.out.println("Vehicle created"); }
}

class Car extends Vehicle {
    Car() { this(4); System.out.println("Car()"); }
    Car(int wheels) { super(); System.out.println("Car with " + wheels + " wheels"); }
}

new Car();
```

Output: Vehicle created Car with 4 wheels Car()

Order to remember: Constructors always execute top-down through the inheritance chain — the topmost superclass constructor finishes first, then each subclass constructor runs, ending with the constructor of the actual class being instantiated. Field initializers run right before each class's own constructor body executes.

7. The `this` and `super` Keywords

7.1 `this`

- Refers to the current object instance — commonly used to disambiguate a field from a same-named parameter.
- Can pass the current object as an argument to another method or constructor.
- Can chain to another constructor of the same class: `this(...)`.

```
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data; // disambiguates field from parameter
        this.next = null;
    }
}
```

7.2 `super`

- Calls the immediate parent's constructor: `super(...)`.
- Accesses a parent's overridden method: `super.methodName()`.
- Accesses a parent's field that is shadowed/hidden by a subclass field of the same name.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override
    void sound() {
        super.sound(); // call the parent's version too
        System.out.println("Bark");
    }
}
```

8. No Destructors — Garbage Collection & try-with-resources

Java has no destructor. Memory is reclaimed automatically by the **Garbage Collector (GC)**, so there is no manual deallocation equivalent to C++'s `delete`.

finalize() — deprecated: Java used to offer a `finalize()` method the GC could call before reclaiming an object, but it has been **deprecated since Java 9** and its timing is never

guaranteed (it may run late, or not at all before JVM shutdown). It should not be relied upon. The modern, deterministic replacement for cleanup logic — closing files, sockets, database connections — is the **try-with-resources** statement, used with any class implementing `AutoCloseable`.

```
class Resource implements AutoCloseable {
    Resource() { System.out.println("Constructor in use"); }
    @Override
    public void close() { System.out.println("Cleanup in use"); }
}

public class Main {
    public static void main(String[] args) {
        try (Resource a = new Resource(); Resource b = new Resource()) {
            // use resources
        } // close() is called automatically, in REVERSE order of creation
    }
}
```

Output: Constructor in use Constructor in use Cleanup in use Cleanup in use

Advanced note: resources declared in a try-with-resources header are closed in the *reverse* order of their declaration (like a stack) — the last resource opened is the first one closed. This matters when resources depend on each other (e.g. a buffered stream wrapping a file stream).

9. Friend Functions & Aggregation

9.1 No "friend" in Java

In C++, a `friend` function can access a class's private/protected members without being a member itself. Java has **no such concept** — allowing arbitrary external functions special access would undermine the strict encapsulation Java enforces. Java achieves similar effects legitimately by:

- Placing related classes in the same **package** and using default (package-private) access.
- Exposing controlled access via public getters/setters.
- Using a **nested (inner) class**, which can directly access the enclosing class's private members.

9.2 Aggregation

Aggregation is when one class holds a reference to another as a field — a **"HAS-A"** relationship — used to reuse a class without inheriting from it.

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Employee {
    String name;
    Address address; // Employee HAS-A Address
    Employee(String name, Address address) {
        this.name = name;
        this.address = address;
    }
}
```

Aggregation vs Composition (a finer distinction): both are "HAS-A", but composition implies *ownership* — the contained object's lifecycle is tied to the container (e.g. a `House` and its `Room` s: rooms don't outlive the house). Aggregation is a looser association where the contained object can exist independently (e.g. a `University` and its `Student` s: students exist before and after enrollment).

10. Abstract Classes & Interfaces

10.1 Abstract Methods and Classes

An **abstract method** has no body — just a signature followed by a semicolon — and serves purely as a placeholder that concrete subclasses must implement. A class containing at least one abstract method must itself be declared `abstract`, and cannot be instantiated directly.

```
abstract class Shape {
    abstract void draw(); // no body
    void describe() { // abstract classes CAN have concrete methods too
        System.out.println("I am a shape.");
    }
}

class Rectangle extends Shape {
    void draw() { System.out.println("Rectangle"); }
}
```

10.2 Interfaces

An interface behaves like a fully abstract contract. Since **Java 8**, interfaces may also declare:

- `default` methods — with a body, providing a default implementation that implementing classes can inherit or override.
- `static` methods — utility methods called on the interface itself.
- Since **Java 9**: `private` methods — helper methods shared between default methods, not visible outside the interface.

All interface fields are implicitly `public static final` (constants). A class can `implements` any number of interfaces — this is Java's substitute for multiple class inheritance.

```
interface Greetable {
    String getName(); // abstract
    default void greet() { // default, has a body
        System.out.println("Hello, " + getName());
    }
    static Greetable of(String n) { // static factory method
        return () -> n; // lambda implementing the single abstract method
    }
}
```

10.3 Abstract Class vs Interface — the decision table

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract, default, static, private (Java 8/9+)
Fields	Any (instance/static, any modifier)	Only <code>public static final</code> constants
Constructors	Yes	No
Inheritance	A class can extend only one abstract class	A class can implement many interfaces
Access modifiers on methods	Any	Implicitly public (except private helpers)
Use when...	Classes share common state/behavior and a clear IS-A hierarchy	Unrelated classes need to guarantee a capability/contract

Functional interfaces & lambdas: an interface with exactly one abstract method is a **functional interface** (e.g. `Runnable`, `Comparator<T>`, `java.util.function.Function<T,R>`). These can be implemented inline with a **lambda expression** instead of a full anonymous class, which is the foundation of Java's functional-style Streams API. The optional `@FunctionalInterface` annotation asks the compiler to enforce that exactly one abstract method exists.

11. Packages & Access Modifiers

A **package** is Java's equivalent of a C++ namespace — a logical (and physical, directory-based) grouping of related classes that avoids naming collisions and organizes code (e.g. `java.util`, `java.io`). `import java.util.*;` brings classes from that package into scope.

Modifier	Same class	Same package	Subclass (different package)	World
<code>private</code>	✓	✗	✗	✗
<code>default</code> (no modifier)	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

This four-level system is one more level of granularity than C++ offers — the *default*

12. Advanced & Frequently-Tested Topics

12.1 The `Object` Class — `equals()`, `hashCode()`, `toString()`

Every class in Java implicitly extends `java.lang.Object` if it doesn't extend anything else — so every object automatically has methods like `equals()`, `hashCode()`, `toString()`, and `clone()`.

- `==` vs `equals()`:** `==` compares references (memory addresses) for objects, or raw values for primitives. `equals()` is meant to compare *logical* equality and can be overridden (e.g. `String` overrides it to compare character content).
- The `equals/hashCode` contract:** if two objects are equal according to `equals()`, they *must* return the same `hashCode()`. Breaking this contract silently corrupts hash-based collections like `HashMap` and `HashSet` (lookups can "lose" objects that are logically present). The reverse isn't required — unequal objects *may* share a hash code (a "collision"), which is allowed and handled internally.

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() { return Objects.hash(x, y); }
}
```

12.2 Object Cloning — Shallow vs Deep Copy

Implementing `Cloneable` (a **marker interface** — one with no methods at all, just a flag) and overriding `clone()` lets you duplicate an object. The default `Object.clone()` does a **shallow copy**: primitive fields are copied by value, but reference fields (arrays, other objects) are copied by reference — both the original and the clone point to the *same* nested object. A **deep copy** requires manually cloning nested mutable objects too.

12.3 Inner (Nested) Classes

Kind	Description
Static nested class	Declared <code>static</code> inside another class; behaves like a top-level class scoped by name, no implicit reference to an outer instance.
Inner (non-static) class	Tied to an instance of the outer class; holds an implicit reference to it and can access its private members.
Local class	Defined inside a method body; visible only within that method.
Anonymous class	A one-off, unnamed class defined and instantiated in a single expression — commonly used before lambdas existed to implement listeners/callbacks inline.

12.4 Enums Are Classes Too

A Java `enum` is a special kind of class where each constant is a `public static final` instance of the enum type. Enums can have fields, constructors, and methods, and can even override methods per-constant.

```
enum Planet {
    MERCURY(3.3e23), EARTH(5.9e24);
    final double mass;
    Planet(double mass) { this.mass = mass; }
}
```

12.5 Generics & Type Erasure

Generics (`List<String>`) give compile-time type safety and remove the need for manual casting. Java implements generics via **type erasure**: generic type information exists only at compile time for checking; at runtime, `List<String>` and `List<Integer>` are both just `List` — the compiler inserts the necessary casts. This is why you cannot do `new T[10]` or `instanceof List<String>` inside generic code.

12.6 Records (Java 16+) and Sealed Classes (Java 17+)

- Records** — `record Point(int x, int y) {}` — are a concise way to declare an immutable data-carrier class. The compiler auto-generates a canonical constructor, private final fields, accessor methods, plus sensible `equals()`, `hashCode()`, and `toString()`.
- Sealed classes/interfaces** — `sealed class Shape permits Circle, Square {}` — let a superclass explicitly restrict which classes are allowed to extend/implement it, enabling exhaustive `switch` pattern matching over a closed hierarchy.

12.7 Checked vs Unchecked Exceptions (OOP hierarchy view)

`Throwable` is the root of Java's exception hierarchy, splitting into `Error` (serious JVM-level problems, not meant to be caught) and `Exception`. `RuntimeException` and its subclasses are **unchecked** (not enforced by the compiler); everything else under `Exception` is **checked** — the compiler forces you to either catch it or declare it with `throws`. This is itself an application of inheritance and polymorphism: a `catch (Exception e)` block can catch any subtype polymorphically.

12.8 SOLID Principles (OOP design, frequently asked in senior interviews)

Principle	Idea
S — Single Responsibility	A class should have only one reason to change.
O — Open/Closed	Open for extension, closed for modification — add behavior via new subclasses/implementations, not by editing existing tested code.
L — Liskov Substitution	Subclass objects must be replaceable for their superclass without breaking correctness.
I — Interface Segregation	Prefer many small, specific interfaces over one large, general-purpose one.
D — Dependency Inversion	Depend on abstractions (interfaces), not concrete implementations.

12.9 Common Design Patterns Built on OOP

- Singleton** — a private constructor plus a static accessor ensures exactly one instance exists (often asked alongside: "*can a constructor be private? Why?*" — yes, precisely for this pattern, and for utility classes with only static members).
- Factory** — a method that returns objects of a common supertype, hiding which concrete subclass is instantiated — pure polymorphism in action.
- Builder** — constructs complex immutable objects step by step, avoiding telescoping constructors.
- Observer** — objects (observers) subscribe to state changes in another object (the subject); the basis of most GUI event-handling and reactive systems.
- Strategy** — encapsulates interchangeable algorithms behind a common interface, injected at runtime — a direct, practical use of polymorphism + composition-over-inheritance.

12.10 instanceof , Downcasting & Pattern Matching

`instanceof` checks an object's actual runtime type before an unsafe downcast (casting a base-type reference down to a more specific subtype). Since Java 16, pattern matching lets you combine the check and the cast in one step:

```
if (obj instanceof Dog dog) {    // "dog" is auto-cast and in scope here
    dog.bark();
}
```

12.11 Marker Interfaces

A marker interface has no methods; it simply "tags" a class so that other code (often via reflection) can check `instanceof` against it and change behavior. Classic examples: `Serializable` (tells the JVM an object may be converted to a byte stream), `Cloneable` (permits `Object.clone()` to work without throwing `CloneNotSupportedException`), and `Remote` (RMI).

Operator Overloading: Java deliberately does not support user-defined operator overloading (unlike C++). The only operator that behaves as "overloaded" in the language itself is `+`, which does both numeric addition and `String` concatenation. This was a design choice to keep code predictable and avoid the readability problems operator overloading can cause in C++.

13. Quick-Reference: Java vs C++ OOP Differences

Feature	C++	Java
Object allocation	Stack or heap	Always heap (via <code>new</code>)
Memory deallocation	Manual (<code>delete</code>)	Automatic (Garbage Collector)
Destructors	Yes	No (use try-with-resources / AutoCloseable)
Multiple class inheritance	Allowed	Not allowed (use interfaces)
Virtual keyword	Required to enable overriding	All non-static/final/private methods are virtual by default
Operator overloading	Supported	Not supported (except built-in <code>+</code>)
Friend functions	Supported	Not supported
Access levels	3 (private, protected, public)	4 (adds package-private/default)
Copy constructor	Built-in, auto-generated	Not built-in; user-defined or <code>clone()</code>
Return type overloading	Also not sufficient alone	Also not sufficient alone (same rule)

14. Interview Questions & Answers

Foundational questions first, followed by advanced/senior-level questions — a comprehensive bank for last-minute revision.

14.1 Core Concepts

1. What are the four main pillars of OOP?	Encapsulation, Abstraction, Inheritance, and Polymorphism.
2. Why is Java called "partially" object-oriented?	Because it still uses primitive types (int, char, double, etc.) that are not objects, unlike a purely object-oriented language where everything is an object.
3. Difference between overloading and overriding?	Overloading: same method name, different parameters, same class, resolved at compile time. Overriding: same signature, parent and child class, resolved at runtime.
4. Why doesn't Java support multiple inheritance of classes?	To avoid the Diamond Problem — ambiguity when a class would inherit conflicting implementations of the same method from two parents. Java uses interfaces instead.
5. How does Java achieve multiple-inheritance-like behavior?	Through interfaces — a class can implement several interfaces; since Java 8, interfaces can have default methods, with explicit resolution rules for conflicts.
6. Abstract class vs Interface?	Abstract class: abstract + concrete methods, constructors, instance variables, single inheritance. Interface: (traditionally) abstract methods only, no constructors, multiple implementation allowed; since Java 8/9, also default, static, private methods.
7. Can we instantiate an abstract class?	No — it must be subclassed, and the subclass must implement all abstract methods or itself be abstract.
8. What is dynamic method dispatch?	The mechanism where a call to an overridden method is resolved at runtime based on the object's actual type, not the reference type — how Java implements runtime polymorphism.
9. Why doesn't Java have destructors?	Because Java automatically manages memory via Garbage Collection. The deprecated finalize() offered similar unreliable functionality; try-with-resources + AutoCloseable is the modern replacement for deterministic cleanup.
10. What is encapsulation and how is it implemented?	Bundling data and methods together and restricting direct access to internal state — implemented via private fields with public getter/setter methods.
11. Method overloading vs constructor overloading?	Constructor overloading is overloading applied specifically to constructors — multiple constructors with different parameter lists.
12. What is the "this" keyword used for?	Refers to the current object — disambiguates fields from parameters, passes the current object to a method, or chains to another constructor via this(...).
13. What is the "super" keyword used for?	Refers to the immediate parent class — calls the parent constructor (super(...)), accesses an overridden method, or a hidden field.
14. Can a static method be overridden?	No — static methods are hidden, not overridden, if redefined in a subclass; resolved at compile time by reference type, unlike true overriding.
15. Difference between "IS-A" and "HAS-A"?	IS-A represents inheritance (Dog IS-A Animal). HAS-A represents composition/aggregation (Car HAS-A Engine).
16. Does Java support operator overloading?	No, except the built-in overload of "+" for both numeric addition and String concatenation.
17. Composition vs inheritance?	Inheritance: IS-A via extends, code reuse through a hierarchy. Composition: HAS-A via containing a reference, offering more flexibility and looser coupling ("favor composition over inheritance").
18. What are Java's access modifiers?	private, default (package-private), protected, public — controlling visibility of classes, fields, methods.
19. Can a constructor be private? Use case?	Yes — prevents object creation from outside the class; used in the Singleton pattern and utility classes with only static members.
20. What happens if a subclass doesn't implement all abstract methods?	The subclass itself must be declared abstract, otherwise it's a compile-time error.

14.2 Advanced & Trickier Questions

21. Contract between equals() and hashCode()?	If two objects are equal via equals(), they must return the same hashCode(). Violating this breaks HashMap/HashSet lookups. Unequal objects may still share a hash code (a collision), which is fine.
22. == vs equals()?	== compares references (identity) for objects or raw values for primitives. equals() compares logical equality and can be overridden (e.g. String compares content, not identity).
23. What is a covariant return type?	When an overriding method returns a subtype of the return type declared in the overridden superclass method — allowed since Java 5, still counts as valid overriding.
24. Shallow copy vs deep copy?	Shallow copy (default Object.clone()) copies primitive fields by value but reference fields by reference, so nested objects are shared. Deep copy recursively clones nested mutable objects too.
25. What is a marker interface? Give examples.	An interface with no methods that simply tags a class for special treatment, usually checked via instanceof/reflection. Examples: Serializable, Cloneable, Remote.
26. What is type erasure in generics?	Java generics exist only at compile time for type checking; at runtime, generic type parameters are erased, so List<String> and List<Integer> share the same runtime class List.
27. What is a functional interface?	An interface with exactly one abstract method (default/static methods don't count), which can be implemented with a lambda expression, e.g. Runnable, Comparator<T>.
28. Static nested class vs inner class?	A static nested class behaves like a top-level class scoped by name and has no implicit reference to an outer instance. A (non-static) inner class holds an implicit reference to its enclosing instance and can access its private members.
29. What is the diamond problem, concretely, and how do default methods reintroduce it?	If two interfaces provide conflicting default implementations of the same method and a class implements both, Java refuses to guess — it's a compile-time error forcing the class to override the method explicitly (e.g. calling Interface.super.method()).
30. Why can't you create a generic array, e.g. new T[10]?	Because of type erasure — at runtime the JVM would not know what element type to enforce, so arrays (which are reified and check element types at runtime) can't safely be created directly for a type parameter.
31. Explain the Liskov Substitution Principle with an OOP example.	Subtypes must be substitutable for their base type without altering correctness. Classic violation: a Square extending Rectangle that overrides setWidth/setHeight to keep both equal — breaks code that assumes setting width alone doesn't change height.
32. What's the difference between a checked and unchecked exception, and how does it relate to OOP?	Checked exceptions (subclasses of Exception, excluding RuntimeException) must be declared or caught, enforced at compile time; unchecked (RuntimeException and subclasses) are not. A catch block can polymorphically catch any subtype of the declared exception class.
33. What is the difference between aggregation and composition?	Both are HAS-A relationships, but composition implies ownership — the contained object's lifecycle depends on the container (a House and its Rooms). Aggregation is a looser association where the contained object can exist independently (a University and its Students).

34. Can an interface extend multiple interfaces?	Yes — unlike classes, interfaces can extend multiple other interfaces, because interfaces only declare contracts (and now defaults), so there's less ambiguity risk, and any conflicts still force explicit resolution in the implementing class.
35. What is the Liskov-adjacent question: can an overriding method throw broader exceptions?	No — an overriding method can only throw the same checked exceptions, a subset of them, or narrower (subclass) checked exceptions, or none at all. It can throw any unchecked exception freely.
36. What is a record, and why does it matter for OOP?	A record (Java 16+) is a concise, immutable data-carrier class where the compiler auto-generates the constructor, accessors, equals(), hashCode(), and toString() — reducing boilerplate for classes that primarily model data rather than behavior.
37. What are sealed classes used for?	They restrict which classes may extend/implement a type (via a permits clause), enabling exhaustive, compiler-checked handling of a closed set of subtypes — useful with pattern matching in switch expressions.
38. Why is String immutable in Java, and how does that relate to OOP design?	Immutability makes String instances safe to share (e.g. via the string pool), thread-safe by default, and safe as HashMap keys since their hashCode never changes — a deliberate encapsulation/design choice, not an accidental limitation.
39. What is the difference between abstraction achieved via abstract classes vs interfaces, practically?	Use an abstract class when subclasses share common state and a natural single-parent hierarchy; use an interface when unrelated classes need to guarantee a capability without being forced into one hierarchy — supporting multiple interface implementation.
40. What is method resolution order when both overloading and overriding are involved (e.g. an overloaded method is also overridden)?	Overload resolution (which method signature to call) happens first, at compile time, based on the static argument types. Once the specific method signature is fixed, overriding resolution (which class's implementation actually runs) happens at runtime, based on the object's real type.